



StitchLink

Premium Embroidery Management System

Full Project Documentation · Technical Deep Dive · 2026

React 18

TypeScript

Vite

Tailwind CSS

Gemini AI

Recharts

Generated: 16 June 2026

1. Project Overview

StitchLink is a modern, full-stack Single Page Application (SPA) designed to bridge the gap between traditional embroidery craftsmanship and digital convenience. It provides a seamless experience for customers to browse visual catalogs, upload and preview embroidery machine code (DST files), and order custom designs. It also offers a powerful, animated admin portal for comprehensive business management.

What Problem Does It Solve?

Boutique embroidery businesses in India struggle with order tracking, design approvals, and inventory management. StitchLink digitizes these workflows — from browsing designs and placing orders to real-time admin analytics and embroidery file parsing.

2. Technology Stack

Frontend Framework: React 18 with TypeScript — component-based UI with static type safety.

Build Tool: Vite — extremely fast development server with Hot Module Replacement (HMR). Much faster than Create React App.

Styling: Tailwind CSS v4 with custom animations (animate-gradient-wave, animate-float-up, animate-grid-pulse, animate-ken-burns) and glassmorphism effects.

Icons: Lucide React — consistent, scalable SVG icons throughout the app.

State Management: React Context API (StoreContext + ThemeContext) — global state without Redux.

Routing: react-router-dom v7 — HashRouter for client-side routing with protected route guards.

AI Integration: Google Gemini API (@google/genai) — natural language design search assistant.

Data Visualization: Recharts — Bar charts and Donut/Pie charts for the admin dashboard.

Deployment Config: vercel.json — configured for Vercel deployment.

3. Application Workflows

3.1 Customer Journey

Step 1: Home Page

Visitor lands on the marketing page. Sees hero, services, testimonials, and Google Maps embed.

Step 2: Login / Signup

Clicks "Get Started". Auth form toggles between Login & Signup mode. Role is assigned based on email.

Step 3: Catalog Page

Browses designs in a responsive grid. Uses the search bar (real-time filter) or category pills. Can open the AI Design Assistant modal.

Step 4: AI Search

Types a natural language query !' Gemini API processes it with the full catalog !' Returns matching design IDs !' Displayed as clickable results.

Step 5: Product Detail

Views an embroidery design with a live fabric color preview (CSS mix-blend-multiply). Selects quantity, adds to cart.

Step 6: Cart Page

Reviews all cart items, sees total. Removes items if needed. Proceeds to Payment.

Step 7: Payment Page

Chooses Card/UPI/COD. Fills card form with live 3D card preview. Submits !' 2.5s simulated gateway !' confetti success animation !' redirected to Profile.

Step 8: Profile Page

Views order history filtered by their user ID. Sees order status badges.

Step 9: Reviews Page

Reads community reviews. Submits their own via a modal form with interactive star rating.

Step 10: DST Uploader

Uploads a Tajima DST binary file !' in-browser parser !' HTML Canvas stitch preview.

3.2 Admin Journey

Step 1: Admin Login

Navigates to /admin/login. Logs in with email containing "admin" or the owner email.

Step 2: Admin Dashboard

Sees 4 KPI cards (Revenue, Orders, Avg Value, Pending). Views Weekly Revenue Bar Chart. Views Order Status Donut Chart. Sees last 5 orders.

Step 3: Admin Orders

Full order list. Can update status: PENDING !' PROCESSING !' SHIPPED !' DELIVERED.

Step 4: Admin Inventory

Manages design catalog. Can edit prices, add new designs via form.

4. DST File Uploader — Technical Deep Dive

This is the most technically advanced feature in StitchLink. It implements a complete binary file parser for the Tajima DST embroidery machine format, entirely in the browser using the Web FileReader API — with no external parsing library.

What is a DST File?

- DST = Data Stitch Tajima — the industry-standard machine code for embroidery machines.
- Binary format: NOT human-readable. Each instruction is encoded in 3 bytes.
- First 512 bytes = ASCII header (metadata like design name).
- From byte 512 onwards = stitch data records (3 bytes each).

Parsing Logic — Step by Step

Step 1: Header Parsing (Bytes 0–511)

TextDecoder reads the first 512 bytes as ASCII. The label is extracted by looking for the "LA:" tag using a regex: `/LA:(.+?)[\r\n\x00]/`

Step 2: 3-Byte Stitch Record Decoding

For each 3-byte group from offset 512:

- Bytes are labeled b0, b1, b2.
- X movement = `getX(b2, b1, b0)` — uses bitwise AND (&) and bit-shift (<<) to decode packed values.
- Y movement = `getY(b2, b1, b0)` — same technique for Y axis.
- Flag check: `(b2 & 0xC0) === 0xC0` !' COLOR_CHANGE (new thread)
- Flag check: `(b2 & 0x0C) === 0x0C` !' JUMP (needle lifts, moves without stitching)
- Neither flag !' STITCH (normal needle stitch, draws a line)
- End marker: `(b2 & 0b11110011) === 0b11110011` !' END of pattern

Step 3: Dimension Calculation

Tracks minX, maxX, minY, maxY across all STITCH records. DST uses 0.1mm units, so: `widthMm = (maxX - minX) / 10`

Step 4: HTML Canvas Rendering

- All stitch coordinates are scaled to fit the 600×500 canvas.
- Y-axis is flipped (DST Y-axis is inverted relative to Canvas Y-axis).
- STITCH records draw lines using `ctx.beginPath()`, `ctx.moveTo()`, `ctx.lineTo()`, `ctx.stroke()`.
- COLOR_CHANGE records switch the current stroke color from a thread palette array.
- JUMP records move the "pen" position without drawing.

5. State Management — StoreContext

There is no Redux. All global state lives in a single React Context (`StoreContext.tsx`). It wraps the entire app and exposes state + action functions to every page via the `useStore()` hook.

State Variables

designs (`Design[]`) — All embroidery designs. Loaded from `mockData.ts` on app start.

orders (`Order[]`) — All orders placed in the current session (in memory).

currentUser (`User | null`) — The logged-in user object (name, email, role: CUSTOMER|ADMIN).

cart (`CartItem[]`) — Items currently in the shopping cart.

reviews (Review[]) — All design reviews. Pre-populated from mockData.ts.

notifications (Notification[]) — Admin notifications triggered when a new order is placed.

Key Action Functions

- login(email, password) — Mock auth. Sets currentUser in state. Role determined by email string.
- signup(name, email, password) — Creates a new CUSTOMER user in memory.
- logout() — Clears currentUser, cart, and notifications.
- addToCart(item) / removeFromCart(id) / clearCart() — Cart management.
- placeOrder() — Converts cart into an Order object, saves it to state, creates a Notification.
- updateOrderStatus(orderId, status) — Admin updates an order's lifecycle status.
- addDesign(design) — Admin adds a new design to the catalog.
- addReview(review) — Customer submits a review, instantly appears in the list.

6. Security & Authentication

Note: The current implementation uses mock authentication (no real backend/database). The architecture is designed to easily swap mock functions with real API calls (Supabase, Firebase, or custom Node.js backend).

Role-Based Route Protection

Every customer and admin page is wrapped in a `<ProtectedRoute>` component in `App.tsx`. This component reads `currentUser` from `StoreContext` and applies two checks:

- If `currentUser` is null `!` redirect to `/login`.
- If `currentUser.role` does not match the page's `allowedRole` `!` redirect to `/`.

This means customers cannot access `/admin` pages, and admins cannot browse the customer `/catalog`.

Environment Variables

- `VITE_GEMINI_API_KEY` is stored in `.env.local` (not committed to Git via `.gitignore`).
- Accessed in code as `import.meta.env.VITE_GEMINI_API_KEY` (Vite's env variable system).

7. UI Design System & Animations

Core Design Principles

- Glassmorphism: `backdrop-blur-xl + bg-white/80` = frosted glass card effect throughout the app.
- Dark/Light Mode: Full support via `ThemeContext`. Users can toggle; system preference is respected.
- Responsive: Tailwind's responsive prefixes (`sm:`, `md:`, `lg:`, `xl:`) ensure mobile-first layouts.

Signature Animations

animate-ken-burns: Home page hero background — slow pan & zoom on the embroidery photo.

animate-blob: Blurred colored circles that float and morph on the home page background.

animate-gradient-wave: Shifting 4-color gradient background across protected pages.

animate-float-up: Tiny particle dots that float upward on catalog, profile, and DST pages.

animate-grid-pulse: Subtle geometric grid that pulses — visible in dark mode on catalog/admin.

animate-thread-dash: SVG thread paths with animated `stroke-dasharray` on the About page.

animate-thread-float: SVG threads that slowly drift horizontally on the About page.

fall (keyframe): Confetti particles falling after successful payment.

8. Project Structure

- `/pages/` — 14 page components (Home, Catalog, Payment, DSTUpload, AdminDashboard, etc.)
- `/components/` — Shared components like Navbar
- `/context/` — `StoreContext.tsx` (global state) + `ThemeContext.tsx` (dark mode)
- `/data/` — `mockData.ts` (sample designs + reviews)
- `/services/` — `geminiService.ts` (Gemini AI API calls)
- `/lib/` — Utility functions
- `/public/` — Static assets (images)
- `App.tsx` — Root: Router, ThemeProvider, StoreProvider, all Routes, Footer
- `types.ts` — TypeScript interfaces (Design, Order, User, CartItem, Review)

- vite.config.ts — Vite build configuration
- vercel.json — Deployment config for Vercel
- .env.local — Secret API keys (not in Git)

StitchLink

Bridging traditional craftsmanship with digital innovation.

Built with [React](#) · [TypeScript](#) · [Vite](#) · [Tailwind CSS](#) · [Google Gemini AI](#)